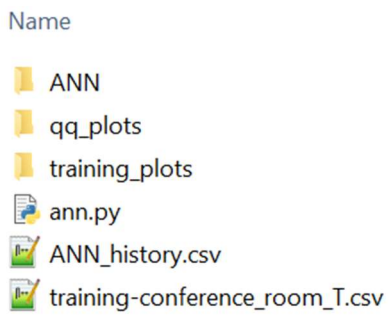
Follow along as you read the code ann.py. This will help you understand what the code is doing and provide helpful tips for setting up the ANN and understanding how it works.

Before you start, make sure the folder that you are running the code in looks like this:



You can have multiple csv's of training data, compared to the one you see above.

ann.py contains all of the relevant code. There are certain parts of the code that are written to help store your work so you can look at it later to improve your understanding and reference it for your report. The "ANN" folder will contain each ANN you train. The "training_plots" folder will have graphs showing the accuracy of your model (on both training and testing data) as it learns.

NOTE: If your code crashes, check each folder and make sure you have the same number of files in each folder. You do not want to train an ANN but not have relevant information recorded.

```
# %% Import Packages
```

This loads packages (sets of extra commands we can give the computer). Nothing needs to be changed here.

```
# %% Set User Inputs
```

"set_name" is the name of the csv file the data is in.

"pred_var" is the variable we are training the ANN to predict. Copy and paste the name from the column in the .csv corresponding to our variable. In this case, the variable is the last (rightmost) one listed in the csv.

Hyperparameters: I recommend reading the linked article [How to tune hyperparameters for better neural network performance] by SangGyu An. I've included a pdf version as well. This is a good article that provides an example of someone trying to optimize the accuracy of an ANN. However, parts of it go outside the scope of this class. If you are having trouble understanding this article, you can simply go to the tables and select the values there as a starting point and vary your values from there. You do not have to code a grid search or implement code like the author does. As you test different values, note how training and the resulting ANN behaves. This should be of interest for your report.

```
# %% Import & Format Data
```

Write code for the two variables, “data” and “target.” Hints for each are included in the comments. Make sure the csv with your data is in the same folder as your input. Using pd.read_csv() loads your data as a dataframe- a dataframe is just a way to store an array of data that is used in the pandas package. It is essentially a matrix with each row and column given a name. A visual example:

	Column 1 Name	Column 2 Name
Row 1 Name	Data entry 1,1	Data entry 1,2
Row 2 Name	Data entry 2,1	Data entry 1,2

Normalization: There are many ways to normalize your data. You can read about what normalization is and how to do it here. To implement in python, use pandas commands such as unfrmtd_data.std() to find the standard deviation of each column. To find other useful pandas commands, simply search “___ of dataframe pandas,” and the first few results will have a satisfactory answer. For example, “maximum of dataframe pandas.” Note that to use the ANN after we train it, the inputs will have to be normalized using the same equation and the outputs will be normalized, so they will need to be reversed to make sense. So, save any values used to normalize data for later. My example uses z-score normalization, so I save the mean and standard deviation.

Note: pandas automatically calculates values in columns, for example, if you ask for the maximum value of unfrmtd_data, the result will be a row of values, with each entry the maximum value of its column. Let’s call the resulting row of values “max.” If you were to divide the unfrmtd_data by max, each column of unfrmtd_data will be divided by the value in the matching row max. This is illustrated below:

$$\max\left(\begin{bmatrix}1 & 5\\ 3 & 8\end{bmatrix}\right) = \begin{bmatrix}3 & 8\end{bmatrix} \qquad \begin{bmatrix}1 & 5\\ 3 & 8\end{bmatrix} / \begin{bmatrix}3 & 8\end{bmatrix} = \begin{bmatrix}1/3 & 5/8\\ 3/3 & 8/8\end{bmatrix}$$

This will make normalization easier.

```
# %% Build ANN
```

Build up your ANN using the layers you’d like. I recommend using dense layers, though TensorFlow has many to choose from, if you wish to try others.

model.summary prints the architecture in readable terms. You may want to copy this information for later.

```
# %% Train ANN
```

Compiling: This takes your ANN model and tells the computer how to train it. “loss” is used to train the ANN, and “metrics” are for the user to monitor the progress of training. You only need to specify the optimizer (I recommend Adam), loss rate, and metric. See here for more on compiling, here for a list of loss functions, and here for a list of metrics.

Training: The ANN is trained using a training data set during each epoch. At the start of each epoch, the dataset is randomly split into a training set and a validation set. The test_set_percentage variable determines what percentage of the data is saved for validation. Consider an example for a dataset with one input and one output:

$$dataset = \begin{bmatrix} 0 & 9 \\ 1 & 2 \\ 2 & 2 \\ 3 & 5 \\ 4 & 7 \\ 5 & 4 \\ 6 & 1 \\ 7 & 4 \\ 8 & 1 \\ 9 & 4 \end{bmatrix}, test\ set\ percentage = 0.2, batch\ size = 2$$

Each epoch, the dataset is split into a test and validation set:

$$test\ set = \begin{bmatrix} 0 & 9 \\ 1 & 2 \\ 2 & 2 \\ 4 & 7 \\ 5 & 4 \\ 6 & 1 \\ 7 & 4 \\ 9 & 4 \end{bmatrix}, \qquad validation\ set = \begin{bmatrix} 3 & 5 \\ 8 & 1 \end{bmatrix}$$

Next, the ANN is trained over a number of steps. Each step starts with a batch of data being selected. A batch is a group of randomly selected entries (rows) from the training data. The variable batch_size indicates how many entries are selected per batch. The selected entries are removed from the training set. Steps are repeated until the training set is empty. For the example dataset:

$$batch_1 = \begin{bmatrix} 7 & 4 \\ 0 & 9 \end{bmatrix} \qquad batch_2 = \begin{bmatrix} 4 & 7 \\ 9 & 4 \end{bmatrix}$$

$$batch_3 = \begin{bmatrix} 2 & 2 \\ 5 & 4 \end{bmatrix} \qquad batch_4 = \begin{bmatrix} 6 & 1 \\ 1 & 2 \end{bmatrix}$$

Every step in the epoch, the inputs from the data in the batch go through the ANN. The resulting values are the ANN's approximation of the target data (the values in the variable "target"). A perfect ANN would be able to output numbers that match the values in the "target" array. These approximated values are compared to the target data using the loss function you selected when compiling the model. The error is used to change the weights of the ANN so that the next guess should be closer. After a certain number of steps, the training epoch finishes. TensorFlow prints something like the screenshot below:

```
Epoch 7/100
262/262 [==============================] - 0s 1ms/step - loss: 0.4205 - mape: 2.2036 - val_loss: 0.4825 - val_mape: 2.3356
Epoch 8/100
262/262 [==============================] - 0s 1ms/step - loss: 0.3857 - mape: 2.1060 - val_loss: 0.5453 - val_mape: 2.5087
```

The ANN from the final step of the epoch is used to calculate the values for "loss," "val_loss," "mape," and "val_mape" that we see in the screenshot. "loss" as seen above is also called training loss. It is calculated by passing the training data through the ANN, and comparing the outputs to the target values using the loss function. "mape" (the training metric) uses the same training data, but compares the outputs to the target values using the function specified as "metrics" when the model was compiled. It is called "mape" in the screenshort because I used the mape (mean average percent error) function as my metric. "val_loss," or validation loss, uses that same ANN and loss function as "loss," but uses the validation data inputs and target values. "val_mape" (validation metric) uses the same ANN and loss

function as “mape,” but with validation data inputs and target values. Ideally, we want our validation loss to equal the training loss and the validation metric to equal the training metric. This is unlikely to happen unless we are overfitting (our ANN is too complex). When these values are close, the ANN can approximate data is has not seen (the validation data) almost as well as the training data. If the values are close and very small, this indicates that the ANN is sufficiently trained to predict whatever we are training it for. However, this can take many epochs to occur.

```
# %% Plot Training Data
```

The y-axis has your loss and metric function values for each epoch. Change plt.legend(['Training: ', 'Validation: '], loc='upper left') to include the functions you used for loss (training) and metric (validation) accordingly. A reflection on how different variations of ANN architecture and hyperparameters change these graphs would be helpful for your report.

```
# %% Calculate Error
```

The only change you need to make is to write the equation for the type of loss that you specified when compiling your ANN. Go back to the description of loss functions, can click on the link to the loss function you selected. An equation describing how it is calculated will be included. To implement that equation in python, use numpy commands such as np.abs(x) to find the absolute value of x. To find other relevant numpy commands, simply search “calculate ___ numpy,” and the first few results will have a satisfactory answer. For example, “calculate mean numpy.”

Save: The remaining code in this section writes information to ANN_history.csv for your records. It would help to keep your own record of the ANN architecture used for each time you run the code. You can run model.summary() to see the architecture.

```
# %% Plot QQ Chart
```

See here for an explanation of qq plots.

Reverse Normalization: The only thing you want to change are the two lines under reverse normalization. They will be the inverse of the normalization function you used to format the data in the “Import and Format Data” section. For example, the reverse of z-score normalization, $x' = (x - \mu)/\sigma$ is $x = (x' * \sigma) + \mu$. You will be using pandas here. To reference the values pertaining to the output from the normalization information that you saved before, use dataframe.iloc[-1], where dataframe is a value used to normalize the data. μ is an example for z-score normalization.